

Experiment 1:

8-Puzzle Problem

Aim:

To write a python program to solve the 8-puzzle problem.

OBJECTIVE :

To understand heuristic-based search techniques and apply the A* algorithm with the Manhattan distance heuristic to solve the 8-puzzle sliding tile problem.

Algorithm:

Step1: Dequeue the state with the lowest priority from the frontier.

Step2: If the dequeued state is the goal state, return the solution path.

Step3: Add the dequeued state to the explored set.

Step4: Generate all possible successor states by swapping the empty tile with its neighboring tiles.

Step5: For each successor state, calculate its priority based on the Manhattan distance and add it to the frontier if it has not been explored before.

Program:

```
from queue import PriorityQueue
```

```
goal = (1, 2, 3,  
        4, 5, 6,  
        7, 8, 0)
```

```
def heuristic(state):  
    h = 0  
    for i in range(9):  
        if state[i] != 0:  
            x1, y1 = divmod(i, 3)  
            x2, y2 = divmod(state[i]-1, 3)  
            h += abs(x1-x2) + abs(y1-y2)  
    return h
```

```
def successors(state):  
    moves = []  
    i = state.index(0)  
    r, c = divmod(i, 3)  
  
    directions = [(-1,0),(1,0),(0,-1),(0,1)]
```

```

for dr, dc in directions:
    nr, nc = r+dr, c+dc
    if 0 <= nr < 3 and 0 <= nc < 3:
        ni = nr*3 + nc
        new = list(state)
        new[i], new[ni] = new[ni], new[i]
        moves.append(tuple(new))

return moves

def solve(start):
    pq = PriorityQueue()
    pq.put((heuristic(start), 0, start, [start]))
    visited = set()

    while not pq.empty():
        f, g, state, path = pq.get()

        if state == goal:
            return path

        if state in visited:
            continue

        visited.add(state)

        for nxt in successors(state):
            if nxt not in visited:
                pq.put((g+1+heuristic(nxt), g+1, nxt, path+[nxt]))

    return None

start = (1, 2, 3,
        5, 6, 0,
        7, 8, 4)

solution = solve(start)

if solution:
    print("Solution Path:\n")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i], state[i+1], state[i+2])
        print()
else:
    print("No Solution")
```

Output:

1 2 3

5 6 0
7 8 4

1 2 3
5 0 6
7 8 4

1 2 3
0 5 6
7 8 4

1 2 3
7 5 6
0 8 4

1 2 3
7 5 6
8 0 4

1 2 3
7 5 6
8 4 0

1 2 3
7 5 0
8 4 6

Result: Hence, the simple python program for 8- puzzle is done